

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería
Carrera de Ingeniería en Software

EV - Unidad I - Aporte Parte Práctica

Asignatura:

Administración de Plataformas

Docente:

Ing. Guerrero Ulloa Gleiston Cicerón

Estudiante:

Luna Mora Ernesto Gregory

Curso:

Sexto Semestre - Ingeniería en Software

Quevedo - Ecuador
2025 - 2026

Índice

1. Introducción y Descripción General	2
2. Arquitectura del Sistema Distribuido	2
2.1. Sincronización por Reloj de Lamport	2
2.2. Algoritmo de Elección de Bully	2
3. Código Fuente Implementado	3
3.1. Clase Cliente.java	3
3.2. Clase Mensaje.java	4
3.3. Clase Nodo.java	5
4. Evidencias y Pruebas de Ejecución	11
4.1. Fase de Autenticación y Envío de Operaciones	11
4.2. Comportamiento de los Nodos y Elección de Líder	11
5. Conclusiones	12

1. Introducción y Descripción General

El presente documento detalla el diseño, desarrollo e implementación de un **Sistema de Registro Logístico Distribuido** tolerante a fallos. El sistema consta de una arquitectura multi-nodo distribuida orientada al paso de mensajes mediante sockets TCP Concurrentes en lenguaje Java.

El núcleo del proyecto radica en resolver problemas fundamentales de los sistemas distribuidos:

- **Exclusión y Ordenamiento:** Uso de Relojes Lógicos de Lamport para asegurar un orden total consistente de los eventos.
- **Disponibilidad y Monitoreo:** Implementación de un mecanismo de latidos (*Heartbeats*) cruzados para la detección de nodos caídos.
- **Consenso y Coordinación:** Implementación del Algoritmo de Elección de Bully para elegir dinámicamente un nuevo líder cuando el coordinador falla.
- **Seguridad:** Capa de autenticación distribuida basada en tokens antes del procesamiento de operaciones de red.

2. Arquitectura del Sistema Distribuido

El clúster está diseñado bajo una topología estática conocida por todos los nodos, compuesta por tres réplicas con IDs 1, 2 y 3, que operan en los puertos TCP 8081, 8082 y 8083 respectivamente.

2.1. Sincronización por Reloj de Lamport

Para evitar colisiones o discrepancias cronológicas en la red, cada nodo actualiza su contador atómico local mediante la regla matemática de Lamport al recibir un mensaje:

$$L_{local} = \text{máx}(L_{local}, L_{recibido}) + 1 \quad (1)$$

Para los eventos locales (como el registro de una transacción), el reloj simplemente se incrementa en +1. El desempate de transacciones simultáneas se resuelve priorizando el ID del nodo de menor denominación.

2.2. Algoritmo de Elección de Bully

Cuando un nodo detecta la ausencia de *heartbeats* del coordinador actual durante más de 5000ms, asume su caída e inicia una elección enviando un mensaje de tipo **ELECTION** a todos los nodos con un ID estrictamente superior. Si no recibe ninguna respuesta (**ANSWER**), el nodo se proclama a sí mismo como nuevo Coordinador y lo notifica (**COORDINATOR**) al resto de los nodos activos con IDs inferiores.

3. Código Fuente Implementado

3.1. Clase Cliente.java

Esta clase actúa como el emisor externo del sistema, permitiendo al usuario ingresar operaciones de logística por consola, validar localmente su token de seguridad y enviarlas a través de sockets TCP a cualquiera de los puertos del clúster.

```
1 package com.uteq.prueba;
2
3 import java.io.ObjectInputStream;
4 import java.io.ObjectOutputStream;
5 import java.net.Socket;
6 import java.util.Scanner;
7
8 public class Cliente {
9     private static final String TOKEN_CORRECTO = "TOKEN-SECRETO-123";
10    private static final int PUERTO_COORDINADOR_INICIAL = 8083; //
        Conecta a N3 de entrada
11
12    public static void main(String[] args) {
13        System.out.println("=== CLIENTE DEL REGISTRO DISTRIBUIDO ===");
14        Scanner scanner = new Scanner(System.in);
15        System.out.print("Ingrese el token de seguridad para operar: ");
16        String tokenIngresado = scanner.nextLine();
17
18        // 1. Fase de Autenticacin
19        if (!autenticarCliente(tokenIngresado)) {
20            System.out.println("[RECHAZADO] Acceso denegado. Cerrando
                cliente.");
21            return;
22        }
23        System.out.println("[OK] Autenticacin aprobada.");
24
25        // 2. Fase de envio de operaciones
26        while (true) {
27            System.out.println("\nEscriba la operacin de logstica (o
                'salir'):");
28            String operacion = scanner.nextLine();
29            if ("salir".equalsIgnoreCase(operacion)) break;
30
31            System.out.print("A qu puerto del Nodo desea enviar la
                operacin? (8081, 8082, 8083): ");
32            int puertoDestino = Integer.parseInt(scanner.nextLine());
33            enviarOperacion(puertoDestino, operacion);
34        }
35        scanner.close();
36    }
37
38    private static boolean autenticarCliente(String token) {
39        try (Socket socket = new Socket("localhost",
                PUERTO_COORDINADOR_INICIAL);
```

```

40      ObjectOutputStream oos = new
        ObjectOutputStream(socket.getOutputStream());
41      ObjectInputStream ois = new
        ObjectInputStream(socket.getInputStream())) {
42
43      // Enviamos con reloj inicial 0
44      Mensaje msgAuth = new Mensaje(Mensaje.Tipo.AUTENTICAR, 99, 0,
        token);
45      oos.writeObject(msgAuth);
46      Mensaje respuesta = (Mensaje) ois.readObject();
47      return respuesta.getTipo() == Mensaje.Tipo.RESPUESTA_OK;
48  } catch (Exception e) {
49      System.err.println("[ERR] No se pudo conectar al nodo para
        autenticar: " + e.getMessage());
50      return false;
51  }
52  }
53
54  private static void enviarOperacion(int puerto, String operacion) {
55      try (Socket socket = new Socket("localhost", puerto);
56          ObjectOutputStream oos = new
            ObjectOutputStream(socket.getOutputStream());
57          ObjectInputStream ois = new
            ObjectInputStream(socket.getInputStream())) {
58
59      // Envia la operacin al nodo
60      Mensaje msgOp = new Mensaje(Mensaje.Tipo.OPERACION, 99, 1,
        operacion);
61      oos.writeObject(msgOp);
62      System.out.println(" [CLIENTE] Enviando: " + operacion);
63      Mensaje respuesta = (Mensaje) ois.readObject();
64      System.out.println(" [NODO RESPONDE] -> " +
        respuesta.getContenido() + " | Reloj de Confirmacin: " +
        respuesta.getRelojLamport());
65  } catch (Exception e) {
66      System.err.println("[ERR] Fall el envo al nodo en el puerto "
        + puerto + ". Posiblemente cado.");
67  }
68  }
69  }

```

3.2. Clase Mensaje.java

Objeto de datos unificado utilizado para estructurar y clasificar toda la mensajería del sistema distribuido a través de un enumerador.

```

1  package com.uteq.prueba;
2
3  import java.io.Serializable;
4
5  public class Mensaje implements Serializable {

```

```

6     private static final long serialVersionUID = 1L;
7
8     public enum Tipo {
9         AUTENTICAR, OPERACION, HEARTBEAT, ELECTION, ANSWER, COORDINATOR,
10        RESPUESTA_OK, RECHAZADO
11    }
12
13    private Tipo tipo;
14    private int idEmisor;
15    private int relojLamport;
16    private String contenido;
17
18    public Mensaje(Tipo tipo, int idEmisor, int relojLamport, String
19        contenido) {
20        this.tipo = tipo;
21        this.idEmisor = idEmisor;
22        this.relojLamport = relojLamport;
23        this.contenido = contenido;
24    }
25
26    public Tipo getTipo() { return tipo; }
27    public int getIdEmisor() { return idEmisor; }
28    public int getRelojLamport() { return relojLamport; }
29    public void setRelojLamport(int relojLamport) { this.relojLamport =
30        relojLamport; }
31    public String getContenido() { return contenido; }
32
33    @Override
34    public String toString() {
35        return String.format(" [Tipo: %s | De: N%d | Lamport: %d |
36            Contenido: %s]",
37            tipo, idEmisor, relojLamport, contenido);
38    }
39 }

```

3.3. Clase Nodo.java

Representa el componente central. Gestiona el servidor multi-hilo, actualiza los relojes lógicos, envía pings periódicos y reacciona ante caídas ejecutando el consenso Bully.

```

1 package com.uteq.prueba;
2
3 import java.io.*;
4 import java.net.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.concurrent.atomic.AtomicInteger;
8
9 public class Nodo {

```

```

10     private final int id;
11     private final int puertoServidor;
12     private final AtomicInteger relojLamport = new AtomicInteger(0);
13
14     private final Map<Integer, Integer> topologia = Map.of(
15         1, 8081,
16         2, 8082,
17         3, 8083
18     );
19
20     private int idCoordinador = 3;
21     private final Map<Integer, Long> ultimosHeartbeats = new
        ConcurrentHashMap<>();
22     private final Set<Integer> nodosActivos =
        ConcurrentHashMap.newKeySet();
23     private boolean enEleccion = false;
24     private final String TOKEN_VALIDO = "TOKEN-SECRETO-123";
25
26     public Nodo(int id) {
27         this.id = id;
28         this.puertoServidor = topologia.get(id);
29
30         topologia.keySet().forEach(nodoId -> {
31             if (nodoId != this.id) {
32                 ultimosHeartbeats.put(nodoId, System.currentTimeMillis());
33                 nodosActivos.add(nodoId);
34             }
35         });
36     }
37
38     public void iniciar() {
39         System.out.println("[LOG] Iniciando Nodo " + id + " en el puerto "
            + puertoServidor);
40         new Thread(this::escucharConexiones).start();
41         new Thread(this::enviarHeartbeatsPeriodicos).start();
42         new Thread(this::monitorearVecinos).start();
43     }
44
45     private void escucharConexiones() {
46         try (ServerSocket serverSocket = new ServerSocket(puertoServidor))
47         {
48             while (true) {
49                 Socket socketCliente = serverSocket.accept();
50                 new Thread(() ->
51                     manejarClienteNodo(socketCliente)).start();
52             }
53         } catch (IOException e) {
54             System.err.println("[ERR] Error en el servidor del Nodo " + id
55                 + ": " + e.getMessage());
56         }
57     }

```

```

55
56 private void manejarClienteNodo(Socket socket) {
57     try (ObjectInputStream ois = new
58         ObjectInputStream(socket.getInputStream());
59         ObjectOutputStream oos = new
60             ObjectOutputStream(socket.getOutputStream())) {
61
62         Mensaje msg = (Mensaje) ois.readObject();
63         int relojRecibido = msg.getRelojLamport();
64         int nuevoReloj = relojLamport.updateAndGet(local ->
65             Math.max(local, relojRecibido) + 1);
66
67         System.out.println(String.format("[RELOJ TOTAL] Nodo %d
68             procesa: %s. Reloj Lamport actualizado a %d", id, msg,
69             nuevoReloj));
70
71         switch (msg.getTipo()) {
72             case HEARTBEAT:
73                 ultimosHeartbeats.put(msg.getIdEmisor(),
74                     System.currentTimeMillis());
75                 if (!nodosActivos.contains(msg.getIdEmisor())) {
76                     System.out.println("[FALLO] Nodo " +
77                         msg.getIdEmisor() + " ha vuelto a la vida.");
78                     nodosActivos.add(msg.getIdEmisor());
79                 }
80                 break;
81             case AUTENTICAR:
82                 if (TOKEN_VALIDO.equals(msg.getContenido())) {
83                     oos.writeObject(new
84                         Mensaje(Mensaje.Tipo.RESPUESTA_OK, id,
85                         relojLamport.get(), "Autenticacin Exitosa"));
86                 } else {
87                     oos.writeObject(new Mensaje(Mensaje.Tipo.RECHAZADO,
88                         id, relojLamport.get(), "Token Inválido"));
89                 }
90                 break;
91             case OPERACION:
92                 relojLamport.incrementAndGet();
93                 System.out.println(String.format("[REGISTRO LOG]
94                     **ORDEN TOTAL CONSISTENTE** -> Lamport: %d, Nodo
95                     Desempate: %d, Operacin: %s",
96                     relojLamport.get(), id, msg.getContenido()));
97                 oos.writeObject(new Mensaje(Mensaje.Tipo.RESPUESTA_OK,
98                     id, relojLamport.get(), "Operacin registrada
99                     exitosamente"));
100                 break;
101             case ELECTION:
102                 System.out.println("[BULLY] Recib ELECTION de Nodo " +
103                     msg.getIdEmisor() + ". Respondiendo ANSWER.");
104                 oos.writeObject(new Mensaje(Mensaje.Tipo.ANSWER, id,
105                     relojLamport.get(), "OK"));

```

```

90         if (!enEleccion) {
91             dispararEleccion();
92         }
93         break;
94     case COORDINATOR:
95         this.idCoordinador = msg.getIdEmisor();
96         this.enEleccion = false;
97         System.out.println("[BULLY] **NUEVO COORDINADOR
98             ESTABLECIDO**: Nodo " + idCoordinador);
99         break;
100     default:
101         break;
102     }
103 } catch (Exception e) {
104     // Manejo silencioso de desconexiones normales
105 }
106
107 private void enviarHeartbeatsPeriodicos() {
108     while (true) {
109         try {
110             Thread.sleep(2000); // Cada 2 segundos
111             for (Map.Entry<Integer, Integer> nodo :
112                 topologia.entrySet()) {
113                 if (nodo.getKey() != this.id &&
114                     nodosActivos.contains(nodo.getKey())) {
115                     enviarMensajeSinRespuesta(nodo.getValue(), new
116                         Mensaje(Mensaje.Tipo.HEARTBEAT, id,
117                             relojLamport.get(), "ping"));
118                 }
119             }
120         } catch (InterruptedException e) {
121             Thread.currentThread().interrupt();
122         }
123     }
124 }
125
126 private void monitorearVecinos() {
127     while (true) {
128         try {
129             Thread.sleep(1000);
130             long ahora = System.currentTimeMillis();
131             for (Integer nodoId : topologia.keySet()) {
132                 if (nodoId != this.id && nodosActivos.contains(nodoId))
133                 {
134                     long ultimoTiempo =
135                         ultimosHeartbeats.getOrDefault(nodoId, ahora);
136                     if (ahora - ultimoTiempo > 5000) { // Tolerancia de
137                         5s
138                         System.out.println(String.format("[ALERTA FALLO]
139                             El Nodo %d no responde heartbeats. Marcado

```

```

        como CAIDO.", nodoId));
        nodosActivos.remove(nodoId);
        if (nodoId == idCoordinador) {
            System.out.println("[BULLY] El Coordinador
                cay. Iniciando proceso de eleccin...");
            dispararEleccion();
        }
    }
}

} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}

}

private synchronized void dispararEleccion() {
    enEleccion = true;
    relojLamport.incrementAndGet();
    System.out.println("[BULLY] Enviando mensajes de ELECTION a nodos
        con ID mayor...");
    boolean recibAnswer = false;

    for (Map.Entry<Integer, Integer> nodo : topologia.entrySet()) {
        if (nodo.getKey() > this.id) {
            try (Socket socket = new Socket()) {
                socket.connect(new InetSocketAddress("localhost",
                    nodo.getValue()), 1500);
                ObjectOutputStream oos = new
                    ObjectOutputStream(socket.getOutputStream());
                ObjectInputStream ois = new
                    ObjectInputStream(socket.getInputStream());

                oos.writeObject(new Mensaje(Mensaje.Tipo.ELECTION, id,
                    relojLamport.get(), "Eleccin"));
                Mensaje respuesta = (Mensaje) ois.readObject();
                if (respuesta.getTipo() == Mensaje.Tipo.ANSWER) {
                    recibAnswer = true;
                }
            } catch (Exception e) {
                // Nodo mayor desconectado
            }
        }
    }

    if (!recibAnswer) {
        idCoordinador = this.id;
        enEleccion = false;
        System.out.println("[BULLY] Eleccin ganada! Yo soy el nuevo
            Coordinador: Nodo " + id);
        for (Map.Entry<Integer, Integer> nodo : topologia.entrySet()) {

```

```

175         if (nodo.getKey() != this.id) {
176             enviarMensajeSinRespuesta(nodo.getValue(), new
                Mensaje(Mensaje.Tipo.COORDINATOR, id,
                relojLamport.get(), "Coordinador"));
177         }
178     }
179 }
180 }
181
182 private void enviarMensajeSinRespuesta(int puertoDestino, Mensaje
    msg) {
183     try (Socket socket = new Socket()) {
184         socket.connect(new InetSocketAddress("localhost",
            puertoDestino), 1000);
185         try (ObjectOutputStream oos = new
            ObjectOutputStream(socket.getOutputStream())) {
186             oos.writeObject(msg);
187         }
188     } catch (Exception e) {
189         // Manejo de desconexiones
190     }
191 }
192
193 public static void main(String[] args) {
194     if (args.length < 1) {
195         System.out.println("Por favor indica el ID del nodo (1, 2 o
            3). Ejemplo: java -cp . com.uteq.prueba.Nodo 1");
196         return;
197     }
198     int idNodo = Integer.parseInt(args[0]);
199     Nodo nodo = new Nodo(idNodo);
200     nodo.iniciar();
201 }
202 }

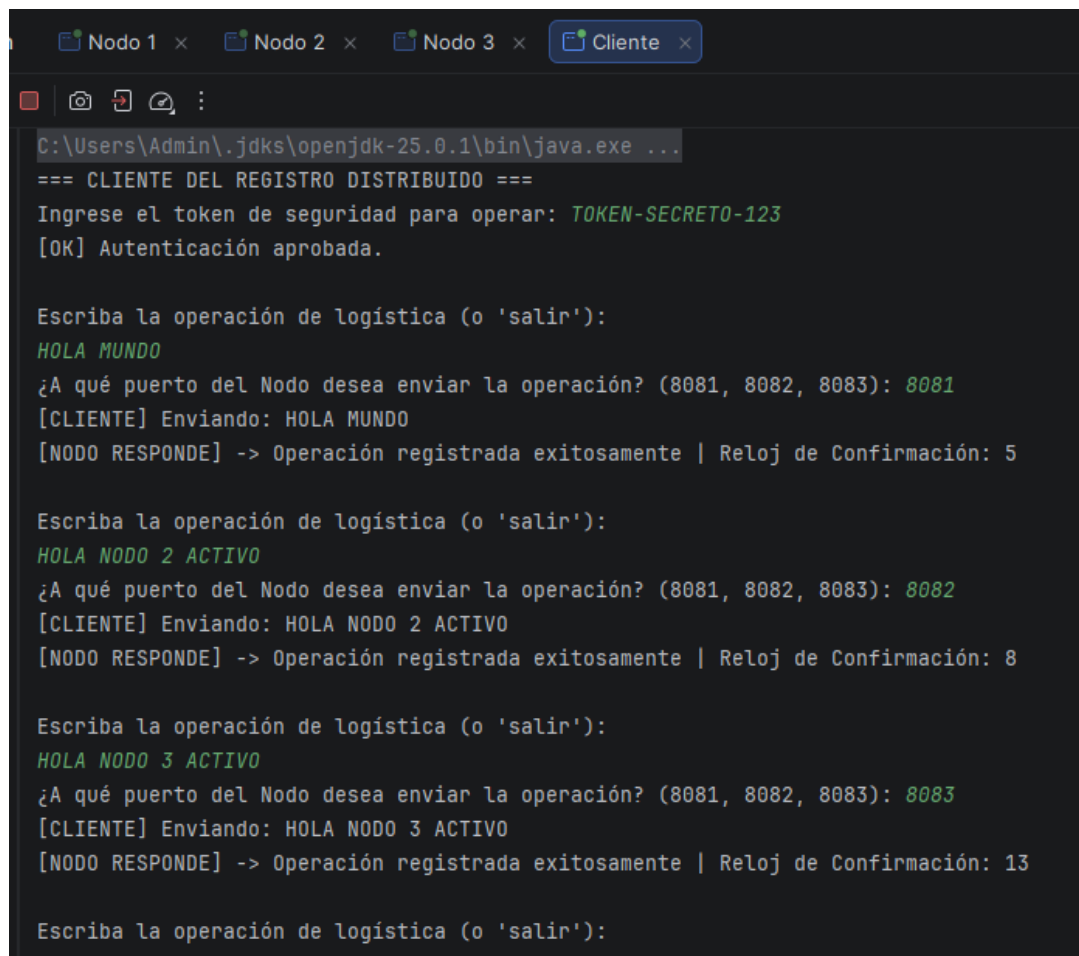
```

4. Evidencias y Pruebas de Ejecución

En esta sección se demuestran los escenarios de ejecución exitosa del sistema, validando el comportamiento dinámico de los hilos de control y la consistencia del log distribuido.

4.1. Fase de Autenticación y Envío de Operaciones

El cliente inicia solicitando el token de seguridad TOKEN-SECRETO-123. Una vez validado por el nodo coordinador inicial, el cliente queda habilitado para enviar cargas de trabajo dirigidas a diferentes puertos del clúster de forma dirigida.



```
C:\Users\Admin\.jdk\openjdk-25.0.1\bin\java.exe ...  
=== CLIENTE DEL REGISTRO DISTRIBUIDO ===  
Ingrese el token de seguridad para operar: TOKEN-SECRETO-123  
[OK] Autenticación aprobada.  
  
Escriba la operación de logística (o 'salir'):  
HOLA MUNDO  
¿A qué puerto del Nodo desea enviar la operación? (8081, 8082, 8083): 8081  
[CLIENTE] Enviando: HOLA MUNDO  
[NODO RESPONDE] -> Operación registrada exitosamente | Reloj de Confirmación: 5  
  
Escriba la operación de logística (o 'salir'):  
HOLA NODO 2 ACTIVO  
¿A qué puerto del Nodo desea enviar la operación? (8081, 8082, 8083): 8082  
[CLIENTE] Enviando: HOLA NODO 2 ACTIVO  
[NODO RESPONDE] -> Operación registrada exitosamente | Reloj de Confirmación: 8  
  
Escriba la operación de logística (o 'salir'):  
HOLA NODO 3 ACTIVO  
¿A qué puerto del Nodo desea enviar la operación? (8081, 8082, 8083): 8083  
[CLIENTE] Enviando: HOLA NODO 3 ACTIVO  
[NODO RESPONDE] -> Operación registrada exitosamente | Reloj de Confirmación: 13  
  
Escriba la operación de logística (o 'salir'):
```

Figura 1: Interconexión global del clúster: Consolas simultáneas en paralelo de los Nodos 1, 2, 3 y la consola de peticiones de entrada del Cliente.

4.2. Comportamiento de los Nodos y Elección de Líder

De acuerdo a las trazas obtenidas de las consolas de los nodos, se verifican con éxito los siguientes estados de la red distribuida:

- **Actualización de Relojes:** El Nodo 1 recibe operaciones e incrementa de forma consistente su reloj de Lamport (reflejando marcas de tiempo coordinadas $\max(local, recibido) + 1$).

- **Detección de Fallos:** Los nodos registran alertas en tiempo real ([ALERTA FALLO]) al interrumpirse los hilos de *pings*.
- **Consenso Bully:** El colapso simulado del Coordinador fuerza la autogestión de elecciones ganadas de forma jerárquica por el Nodo con el ID activo más alto disponible en ese instante.

```
C:\Users\Admin\jdk\openjdk-25.0.1\bin\java.exe ...
[LOG] Iniciando Nodo 1 en el puerto 8081
[ALERTA FALLO] El Nodo 2 no responde heartbeats. Marcado como CAÍDO.
[ALERTA FALLO] El Nodo 3 no responde heartbeats. Marcado como CAÍDO.
[BULLY] El Coordinador cayó. Iniciando proceso de elección...
[BULLY] Enviando mensajes de ELECTION a nodos con ID mayor...
[RELOJ TOTAL] Nodo 1 procesa: [Tipo: HEARTBEAT | De: N3 | Lamport: 0 | Contenido: ping]. Reloj Lamport actualizado a: 2
[FALLO] Nodo 3 ha vuelto a la vida.
[ALERTA FALLO] El Nodo 3 no responde heartbeats. Marcado como CAÍDO.
[BULLY] El Coordinador cayó. Iniciando proceso de elección...
[BULLY] Enviando mensajes de ELECTION a nodos con ID mayor...
[RELOJ TOTAL] Nodo 1 procesa: [Tipo: OPERACION | De: N99 | Lamport: 1 | Contenido: HOLA MUNDO]. Reloj Lamport actualizado a: 4
[REGISTRO LOG] **ORDEN TOTAL CONSISTENTE** -> Lamport: 5, Nodo Desempeate: 1, Operación: HOLA MUNDO

C:\Users\Admin\jdk\openjdk-25.0.1\bin\java.exe ...
[LOG] Iniciando Nodo 2 en el puerto 8082
[RELOJ TOTAL] Nodo 2 procesa: [Tipo: ELECTION | De: N1 | Lamport: 1 | Contenido: Elección]. Reloj Lamport actualizado a: 2
[BULLY] Recibí ELECTION de Nodo 1. Respondiendo ANSWER.
[BULLY] Enviando mensajes de ELECTION a nodos con ID mayor...
[BULLY] **Elección ganada!** Yo soy el nuevo Coordinador: Nodo 2
[RELOJ TOTAL] Nodo 2 procesa: [Tipo: HEARTBEAT | De: N3 | Lamport: 0 | Contenido: ping]. Reloj Lamport actualizado a: 4
[ALERTA FALLO] El Nodo 1 no responde heartbeats. Marcado como CAÍDO.
[ALERTA FALLO] El Nodo 3 no responde heartbeats. Marcado como CAÍDO.
[RELOJ TOTAL] Nodo 2 procesa: [Tipo: ELECTION | De: N1 | Lamport: 3 | Contenido: Elección]. Reloj Lamport actualizado a: 5
[BULLY] Recibí ELECTION de Nodo 1. Respondiendo ANSWER.
[BULLY] Enviando mensajes de ELECTION a nodos con ID mayor...
[RELOJ TOTAL] Nodo 2 procesa: [Tipo: OPERACION | De: N99 | Lamport: 1 | Contenido: HOLA NODO 2 ACTIVO]. Reloj Lamport actualizado a: 7
[REGISTRO LOG] **ORDEN TOTAL CONSISTENTE** -> Lamport: 8, Nodo Desempeate: 2, Operación: HOLA NODO 2 ACTIVO

C:\Users\Admin\jdk\openjdk-25.0.1\bin\java.exe ...
[LOG] Iniciando Nodo 3 en el puerto 8083
[ALERTA FALLO] El Nodo 2 no responde heartbeats. Marcado como CAÍDO.
[ALERTA FALLO] El Nodo 1 no responde heartbeats. Marcado como CAÍDO.
[RELOJ TOTAL] Nodo 3 procesa: [Tipo: ELECTION | De: N2 | Lamport: 6 | Contenido: Elección]. Reloj Lamport actualizado a: 7
[RELOJ TOTAL] Nodo 3 procesa: [Tipo: ELECTION | De: N1 | Lamport: 3 | Contenido: Elección]. Reloj Lamport actualizado a: 8
[BULLY] Recibí ELECTION de Nodo 1. Respondiendo ANSWER.
[BULLY] Recibí ELECTION de Nodo 2. Respondiendo ANSWER.
[BULLY] Enviando mensajes de ELECTION a nodos con ID mayor...
[BULLY] **Elección ganada!** Yo soy el nuevo Coordinador: Nodo 3
[BULLY] Enviando mensajes de ELECTION a nodos con ID mayor...
[BULLY] **Elección ganada!** Yo soy el nuevo Coordinador: Nodo 3
[RELOJ TOTAL] Nodo 3 procesa: [Tipo: AUTENTICAR | De: N99 | Lamport: 0 | Contenido: TOKEN-SECRETO-123]. Reloj Lamport actualizado a: 11
[RELOJ TOTAL] Nodo 3 procesa: [Tipo: OPERACION | De: N99 | Lamport: 1 | Contenido: HOLA NODO 3 ACTIVO]. Reloj Lamport actualizado a: 12
[REGISTRO LOG] **ORDEN TOTAL CONSISTENTE** -> Lamport: 13, Nodo Desempeate: 3, Operación: HOLA NODO 3 ACTIVO
```

Figura 2: Consola de trazas del Nodo 3 interactuando con el algoritmo de exclusión mutua, detectando pérdidas de vecinos y ganando de forma unánime el rol de Coordinador del clúster.

5. Conclusiones

La arquitectura desarrollada demuestra de forma práctica cómo los algoritmos clásicos de sistemas distribuidos resuelven los problemas de sincronización de eventos y alta disponibilidad. Los relojes de Lamport lograron mitigar la falta de un reloj físico global homogéneo, mientras que el algoritmo Bully proveyó al clúster de una capa de auto-sanación indispensable ante fallas imprevistas de infraestructura.